บทที่ 1: ระบบที่เชื่อถือได้ ขยายได้ และดูแลง่าย

(สรุปเป็นภาษาไทยจากฉบับที่ 1 ใช้เวลาฟังประมาณ 12–15 นาที)

ตอนที่ 1: บทนำ

สวัสดีครับ ยินดีต้อนรับสู่บทสรุปหนังสือ Designing Data-Intensive Applications บทที่หนึ่ง

ก่อนอื่น มาดูกันว่าระบบแบบ data-intensive คืออะไร ระบบส่วนใหญ่ในทุกวันนี้ไม่ได้มีปัญหาที่ความเร็วของซีพียู ปัญหาจริงคือข้อมูล ข้อมูลอาจมีมากเกินไป ซับซ้อนเกินไป หรือเปลี่ยนแปลงเร็วเกินไป ระบบที่มีปัญหาแบบนี้เรียกว่าระบบที่เน้นข้อมูล หรือ data-intensive

ระบบแบบนี้มักสร้างจากชิ้นส่วนมาตรฐานไม่กี่อย่าง ได้แก่

- ฐานข้อมูล ใช้เก็บข้อมูลไว้เรียกใช้ภายหลัง
- แคช ใช้จำผลลัพธ์ที่คำนวณยาก เพื่อให้เรียกซ้ำได้เร็วขึ้น
- ระบบค้นหา ใช้ค้นข้อมูลด้วยคำสำคัญหรือเงื่อนไขต่าง ๆ
- ระบบประมวลผลแบบสตรีม ใช้ส่งข้อความระหว่างโปรแกรมแบบทันที
- ระบบประมวลผลแบบแบตช์ ใช้ประมวลผลข้อมูลก้อนใหญ่เป็นรอบ ๆ

ทุกวันนี้เส้นแบ่งระหว่างเครื่องมือเหล่านี้เริ่มไม่ชัดแล้ว บางเครื่องมือทำได้หลายหน้าที่ และหลายระบบก็นำเครื่องมือหลายตัวมาต่อกัน เมื่อเป็นแบบนี้ นักพัฒนาจึงไม่ได้เป็นแค่คนเขียนโปรแกรม แต่ต้องเป็นนักออกแบบระบบข้อมูลด้วย

ผู้เขียนเสนอว่าระบบที่ดีต้องคำนึงถึงสามเรื่องหลัก คือ ความน่าเชื่อถือ ความสามารถในการขยาย และความง่ายในการดูแลรักษา เราจะไปดูทีละเรื่องครับ

[หยุด]

ตอนที่ 2: ความน่าเชื่อถือ หรือ Reliability

ความน่าเชื่อถือ หมายถึง ระบบยังทำงานได้ถูกต้องแม้จะมีบางอย่างผิดพลาด

ในหนังสือแยกคำสองคำออกจากกันอย่างชัดเจน คำแรกคือ fault หรือความผิดพลาดของชิ้นส่วนใดชิ้นส่วนหนึ่ง คำที่สองคือ failure หรือการที่ระบบทั้งระบบหยุดให้บริการผู้ใช้

เราไม่สามารถทำให้ไม่มีความผิดพลาดได้เลย เป้าหมายจริงคือการออกแบบให้ความผิดพลาดเล็ก ๆ ไม่ลุกลามจนระบบทั้งระบบล่ม ระบบแบบนี้เรียกว่า fault-tolerant หรือระบบที่ทนต่อความผิดพลาดได้

มีตัวอย่างที่น่าสนใจคือ บางบริษัทจงใจสร้างความผิดพลาดขึ้นในระบบจริง เช่น สุ่มปิดเครื่องเซิร์ฟเวอร์ทิ้ง เพื่อทดสอบว่าระบบรับมือได้จริงหรือเปล่า เครื่องมือที่มีชื่อเสียงคือ Chaos Monkey ของ Netflix

ความผิดพลาดแบ่งออกได้เป็นสามประเภทหลัก

ประเภทแรก คือ ความผิดพลาดของฮาร์ดแวร์ เช่น ฮาร์ดดิสก์เสีย แรมเสีย หรือไฟดับ ฮาร์ดดิสก์โดยทั่วไปมีอายุเฉลี่ยประมาณ 10 ถึง 50 ปี ฟังดูนาน แต่ถ้าเรามีดิสก์หนึ่งหมื่นลูก ก็อาจมีดิสก์เสียเฉลี่ยวันละหนึ่งลูก วิธีรับมือแบบดั้งเดิมคือทำสำรองไว้ เช่น ใช้ดิสก์หลายลูกแบบ RAID หรือมีไฟฟ้าสำรอง ระบบสมัยใหม่ไปไกลกว่านั้น คือออกแบบให้ซอฟต์แวร์ทำงานต่อได้แม้เครื่องทั้งเครื่องหายไป

ประเภทที่สอง คือ ความผิดพลาดของซอฟต์แวร์ ประเภทนี้อันตรายกว่า เพราะมักเกิดพร้อมกันหลายเครื่อง เช่น บั๊กที่ทำให้ทุกเครื่องล่มเมื่อเจอข้อมูลบางแบบ หรือบริการหนึ่งช้าลงจนบริการอื่นที่พึ่งพามันพังตามกันเป็นลูกโซ่ ประเภทนี้ไม่มีวิธีแก้แบบเบ็ดเสร็จ ต้องอาศัยการทดสอบอย่างละเอียด การแยกส่วนระบบ การเฝ้าระวัง และการตรวจสอบตัวเองอย่างต่อเนื่อง

ประเภทที่สาม คือ ความผิดพลาดของมนุษย์ ซึ่งเป็นสาเหตุหลักของระบบล่ม จากการศึกษาพบว่า ความผิดพลาดในการตั้งค่าระบบโดยผู้ดูแล เป็นสาเหตุอันดับต้น ๆ มากกว่าฮาร์ดแวร์เสียเสียอีก วิธีลดความเสี่ยงมีหลายทาง เช่น

- ออกแบบเครื่องมือให้ทำสิ่งที่ถูกต้องได้ง่าย และทำสิ่งที่ผิดได้ยาก
- มีสภาพแวดล้อมสำหรับทดลอง ที่แยกจากระบบจริง
- ทดสอบอย่างครอบคลุม ทั้งการทดสอบอัตโนมัติและการทดสอบด้วยคน
- ทำให้ย้อนกลับได้ง่ายเมื่อเกิดปัญหา เช่น ค่อย ๆ ปล่อยโค้ดใหม่ให้ผู้ใช้บางส่วนก่อน
- เฝ้าระวังระบบอย่างละเอียด ด้วยตัวเลขสถิติและการแจ้งเตือน

แล้วทำไมความน่าเชื่อถือถึงสำคัญ ไม่ใช่แค่ระบบสำคัญอย่างโรงไฟฟ้าเท่านั้น ลองนึกถึงแอปเก็บรูปของครอบครัว ถ้าฐานข้อมูลเสียแล้วรูปลูกหายไปทั้งหมด ผู้ใช้คงเสียใจมาก ความน่าเชื่อถือจึงเป็นความรับผิดชอบต่อผู้ใช้ด้วย

[หยุด]

ตอนที่ 3: ความสามารถในการขยาย หรือ Scalability

ระบบที่ทำงานได้ดีในวันนี้ ไม่ได้แปลว่าจะทำงานได้ดีในอนาคต ถ้าผู้ใช้เพิ่มจากหนึ่งหมื่นคนเป็นสิบล้านคน ระบบก็อาจรับไม่ไหว

หนังสือย้ำว่า การพูดว่าระบบนี้ขยายได้หรือขยายไม่ได้ ไม่มีความหมายอะไร คำถามที่ควรถามคือ ถ้าภาระงานเพิ่มขึ้นแบบนี้ เราจะรับมืออย่างไร

ขั้นแรก ต้องอธิบายภาระงานให้ได้ก่อน

ภาระงานอธิบายด้วยตัวเลขบางอย่าง เรียกว่า load parameters เช่น จำนวนคำขอต่อวินาที หรืออัตราส่วนระหว่างการอ่านกับการเขียน

ตัวอย่างคลาสสิกในหนังสือคือ Twitter มีงานหลักสองอย่าง อย่างแรกคือการโพสต์ข้อความ ซึ่งมีประมาณสี่พันหกร้อยครั้งต่อวินาทีโดยเฉลี่ย และสูงสุดประมาณหนึ่งหมื่นสองพันครั้ง อย่างที่สองคือการเปิดดูหน้าไทม์ไลน์ ซึ่งมีมากถึงประมาณสามแสนครั้งต่อวินาที

ในการสร้างไทม์ไลน์ มีสองวิธี

วิธีแรก คือ ตอนที่ผู้ใช้เปิดไทม์ไลน์ ระบบค่อยไปดึงโพสต์จากทุกคนที่เขาติดตามมารวมกัน วิธีนี้ทำให้การเขียนง่าย แต่การอ่านหนักมาก

วิธีที่สอง คือ ทันทีที่ใครโพสต์ ระบบจะส่งโพสต์นั้นไปใส่ในไทม์ไลน์ของผู้ติดตามทุกคนไว้ล่วงหน้า เหมือนส่งจดหมายใส่ตู้ไปรษณีย์ของทุกคน วิธีนี้ทำให้การอ่านเร็วมาก เพราะไทม์ไลน์เตรียมไว้แล้ว

Twitter เปลี่ยนมาใช้วิธีที่สอง เพราะคนอ่านมากกว่าเขียนหลายสิบเท่า แต่วิธีนี้มีปัญหากับคนดังที่มีผู้ติดตามหลายสิบล้านคน โพสต์เดียวต้องส่งไปหลายสิบล้านที่ สุดท้าย Twitter จึงใช้ทั้งสองวิธีผสมกัน คนทั่วไปใช้วิธีส่งล่วงหน้า ส่วนคนดังใช้วิธีดึงตอนอ่าน

บทเรียนจากตัวอย่างนี้คือ ตัวเลขที่สำคัญที่สุดของ Twitter ไม่ใช่จำนวนโพสต์ แต่เป็นจำนวนผู้ติดตามของแต่ละคน การรู้ว่าตัวเลขไหนสำคัญ คือหัวใจของการออกแบบ

ขั้นที่สอง ต้องอธิบายประสิทธิภาพให้ถูก

ระบบออนไลน์มักวัดด้วยเวลาตอบสนอง หรือ response time คือเวลาที่ผู้ใช้รอตั้งแต่ส่งคำขอจนได้คำตอบ

เวลาตอบสนองไม่ได้เท่ากันทุกครั้ง แม้จะเป็นคำขอแบบเดียวกัน จึงควรมองเป็นการกระจายของค่า ไม่ใช่ตัวเลขเดียว

หนังสือแนะนำว่า ไม่ควรใช้ค่าเฉลี่ย แต่ควรใช้เปอร์เซ็นไทล์ เช่น ค่ามัธยฐาน หรือเปอร์เซ็นไทล์ที่ 50 หมายความว่า ผู้ใช้ครึ่งหนึ่งได้คำตอบเร็วกว่านี้ อีกครึ่งช้ากว่านี้

ส่วนเปอร์เซ็นไทล์ที่ 95 หรือ 99 จะบอกว่าคนกลุ่มที่โชคร้ายที่สุดต้องรอนานแค่ไหน ค่าเหล่านี้เรียกว่า tail latency หรือความหน่วงช่วงหาง

ตัวอย่างที่น่าสนใจคือ Amazon ให้ความสำคัญกับเปอร์เซ็นไทล์ที่ 99.9 เพราะลูกค้าที่ได้คำตอบช้าที่สุด มักเป็นลูกค้าที่มีข้อมูลการสั่งซื้อเยอะ ซึ่งก็คือลูกค้าที่มีค่ามากที่สุดนั่นเอง

อีกเรื่องที่ควรรู้คือ ถ้าการแสดงหน้าเว็บหนึ่งหน้าต้องเรียกบริการเบื้องหลังหลายตัว ขอแค่ตัวเดียวช้า ทั้งหน้าก็ช้าตาม ยิ่งเรียกหลายตัว โอกาสเจอตัวช้าก็ยิ่งสูง ปรากฏการณ์นี้เรียกว่า tail latency amplification

ขั้นที่สาม วิธีรับมือเมื่อภาระงานเพิ่ม

มีสองแนวทางหลัก

แนวทางแรก คือ ขยายแนวตั้ง หรือ scale up หมายถึงอัปเกรดเป็นเครื่องที่แรงขึ้น

แนวทางที่สอง คือ ขยายแนวนอน หรือ scale out หมายถึงเพิ่มจำนวนเครื่อง แล้วกระจายงานไปหลายเครื่อง

ในทางปฏิบัติ ระบบที่ดีมักใช้ทั้งสองแนวทางผสมกัน บางระบบขยายเองอัตโนมัติเมื่อภาระงานเพิ่ม เรียกว่า elastic

ระบบที่ไม่เก็บสถานะ เช่น เว็บเซิร์ฟเวอร์ทั่วไป กระจายไปหลายเครื่องได้ไม่ยาก แต่ระบบที่เก็บข้อมูล เช่น ฐานข้อมูล กระจายได้ยากกว่ามาก

ข้อสำคัญคือ ไม่มีสถาปัตยกรรมแบบใช้ได้กับทุกระบบ ระบบที่รับคำขอเล็ก ๆ หนึ่งแสนครั้งต่อวินาที กับระบบที่รับคำขอใหญ่ ๆ เพียงสามครั้งต่อนาที ต้องออกแบบต่างกันโดยสิ้นเชิง แม้ปริมาณข้อมูลรวมจะเท่ากันก็ตาม

[หยุด]

ตอนที่ 4: ความง่ายในการดูแลรักษา หรือ Maintainability

ข้อเท็จจริงที่หลายคนมองข้ามคือ ค่าใช้จ่ายส่วนใหญ่ของซอฟต์แวร์ไม่ได้อยู่ที่การสร้างครั้งแรก แต่อยู่ที่การดูแลต่อเนื่อง เช่น แก้บั๊ก ดูแลให้ระบบทำงาน ปรับให้รองรับแพลตฟอร์มใหม่ และเพิ่มความสามารถใหม่

ไม่มีใครชอบดูแลระบบเก่าที่ยุ่งเหยิง เราจึงควรออกแบบตั้งแต่วันแรกให้ดูแลง่าย โดยยึดหลักสามข้อ

ข้อแรก คือ ความง่ายในการปฏิบัติการ หรือ Operability

ทำให้ทีมที่ดูแลระบบทำงานได้สะดวก เช่น มีเครื่องมือเฝ้าดูสถานะระบบที่ชัดเจน ทำงานซ้ำ ๆ ให้เป็นอัตโนมัติ มีเอกสารที่เข้าใจง่าย มีค่าเริ่มต้นที่เหมาะสม และระบบควรทำงานแบบคาดเดาได้ ไม่สร้างเรื่องให้ตกใจ

ข้อที่สอง คือ ความเรียบง่าย หรือ Simplicity

โปรเจกต์ที่ใหญ่ขึ้นมักซับซ้อนขึ้นเรื่อย ๆ จนกลายเป็นก้อนโคลนขนาดใหญ่ที่ไม่มีใครกล้าแตะ

ความซับซ้อนบางส่วนเป็นเรื่องที่หลีกเลี่ยงไม่ได้ เพราะมาจากตัวปัญหาเอง แต่บางส่วนเกิดจากการออกแบบที่ไม่ดีเท่านั้น ส่วนนี้เรียกว่า accidental complexity หรือความซับซ้อนที่ไม่จำเป็น

เครื่องมือสำคัญที่สุดในการลดความซับซ้อน คือ abstraction หรือการซ่อนรายละเอียดไว้ใต้ส่วนเชื่อมต่อที่เรียบง่าย ตัวอย่างเช่น ภาษา SQL ที่ซ่อนความยุ่งยากของการจัดเก็บข้อมูลบนดิสก์และการทำงานพร้อมกันหลายคำขอเอาไว้ ทำให้เราสั่งงานได้ง่าย ๆ ด้วยคำสั่งไม่กี่บรรทัด

ข้อที่สาม คือ ความพร้อมต่อการเปลี่ยนแปลง หรือ Evolvability

ความต้องการของระบบเปลี่ยนแปลงตลอดเวลา เช่น มีกฎหมายใหม่ ธุรกิจเปลี่ยนทิศทาง หรือผู้ใช้ขอความสามารถใหม่ ระบบที่เรียบง่ายและเข้าใจง่าย จะปรับเปลี่ยนได้ง่ายกว่าระบบที่ซับซ้อนเสมอ ความเรียบง่ายกับความพร้อมต่อการเปลี่ยนแปลงจึงไปด้วยกัน

[หยุด]

ตอนที่ 5: สรุปท้ายบท

มาทบทวนกันอีกครั้งครับ

หนึ่ง ความน่าเชื่อถือ คือการทำให้ระบบยังทำงานถูกต้องแม้มีความผิดพลาด ไม่ว่าจะมาจากฮาร์ดแวร์ ซอฟต์แวร์ หรือมนุษย์

สอง ความสามารถในการขยาย คือการมีแผนรับมือเมื่อภาระงานเพิ่มขึ้น เริ่มจากอธิบายภาระงานและวัดประสิทธิภาพให้ถูกต้อง โดยเฉพาะการดูเปอร์เซ็นไทล์แทนค่าเฉลี่ย

สาม ความง่ายในการดูแลรักษา คือการทำให้ชีวิตของวิศวกรที่ต้องดูแลระบบง่ายขึ้น ด้วยการปฏิบัติการที่สะดวก ความเรียบง่าย และความพร้อมต่อการเปลี่ยนแปลง

ไม่มีวิธีลัดที่ทำให้ได้ทั้งสามเรื่องในทันที แต่มีรูปแบบและเทคนิคที่ใช้ซ้ำได้ ซึ่งบทถัด ๆ ไปของหนังสือจะพาไปดูทีละเรื่อง

บทต่อไปคือบทที่สอง ว่าด้วยโมเดลข้อมูลและภาษาสำหรับสืบค้นข้อมูล ขอบคุณที่ฟังครับ